

Week 13: Programs from formal specifications

Functional Programming in Lean

Learning goals

- ▶ Keep the formal statement fixed
- ▶ Review generated code and proof separately
- ▶ Check axiom dependencies after compilation

Central idea

A specification states observable behavior. An implementation and its proof must satisfy that statement.

Lean example

```
theorem partitionBy_yes
  (p :  $\alpha \rightarrow \text{Bool}$ ) (xs : List  $\alpha$ ) :
  (partitionBy p xs).1 = xs.filter p := by
  sorry
```

What to notice

- ▶ A checked proof addresses the formal statement
- ▶ The statement must still match the intended behavior
- ▶ `sorryAx` in an axiom report means the proof is incomplete

Live coding

1. Show the teacher-owned specification
2. Ask the LLM for a program and proofs
3. Audit the response, then compile and check axioms

Pause and predict

Would membership alone specify the order of each output list?

Exercise connection

Open `exercises/Week13.md` after the lecture.

The complete example is `Course/Week13.lean`; the fill-in version is `Templates/Week13.lean`.